

BÁO CÁO BÀI TẬP VỀ NHÀ & CHUẨN BỊ BUỔI 2

BÀI 1: Khảo sát 3 Public API thực tế theo Domain quan tâm

Domain lựa chọn: Hạ tầng Lập trình & Dịch vụ Dữ liệu Đa phương tiện / Tiện ích Toàn cầu.

Dưới đây là khảo sát và phân tích kỹ thuật của 3 Public API đại diện hàng đầu:

Tiêu chí	GitHub REST API	Spotify Web API	OpenWeatherMap API
Domain & Chức năng	Quản lý mã nguồn, repository, pull request, issues, user profile.	Stream nhạc số, catalog bài hát/album/artist, playlist của user.	Dữ liệu thời tiết hiện tại, dự báo, chỉ số ô nhiễm không khí lịch sử.
Loại API	RESTful (kèm song song bản GraphQL v4 riêng biệt).	RESTful.	RESTful (giao tiếp qua HTTP/JSON).
Base URL	https://api.github.com	https://api.spotify.com/v1	https://api.openweathermap.org/data/2.5
Cơ chế Xác thực (Auth Method)	Personal Access Token (PAT), Fine-grained token. Header: Authorization: Bearer <token> hoặc token <token>. Hỗ trợ unauthenticated (bị giới hạn rate-limit 60 req/h).	OAuth 2.0 (Authorization Code Flow, Client Credentials). Header: Authorization: Bearer <access_token>. Bắt buộc auth cho 100% endpoint.	API Key (truyền qua Query Parameter). Ví dụ: ?appid={API_KEY} (hoặc qua custom header x-api-key trên một số gateway).
Versioning	Quản lý qua Header (Calendar Versioning): X-GitHub-API-Version: 2022-11-28	Quản lý trực tiếp trên URI Path: Định danh /v1/ ngay trong Base URL.	Quản lý trực tiếp trên URI Path: Định danh phiên bản như /2.5/, /3.0/ (One Call API 3.0).
Định dạng Resource Identifier	Số nguyên (Integer ID) & Tên/Slug: Ví dụ: user ID là integer (1296269), hoặc dùng slug	Base62 String ID (22 ký tự tương đương mã hash duy nhất):	Số nguyên (City ID) & Tọa độ Geocoding: Ví dụ ID thành phố: 1566083 (Hà Nội), hoặc lục bằng cặp

	/users/{username}, /repos/{owner}/{repo}.	Ví dụ Track ID: 4cOdK2wGLETKB W3PvgPWqT, Spotify URI: spotify:track:4cOd. ...	tọa độ thập phân lat={lat}&lon={lon}.
--	--	--	--

BÀI 2: Phân tích Higginbotham Ch.1 + Geewax Ch.1 & So sánh 5 Đặc điểm API Tốt

1. Tóm lược các định nghĩa cốt lõi theo tác giả

James Higginbotham (Principles of Web API Design - Ch.1): Định nghĩa API là "Giao diện giao tiếp giữa các thành phần phần mềm dựa trên các quy ước thỏa thuận rõ ràng". Với Web API, đó là ranh giới cộng tác kinh doanh số (Digital Collaboration Boundary) cho phép mở rộng hệ sinh thái và giải phóng dữ liệu. Resource được hiểu là một khái niệm mang tính nghiệp vụ (Domain Concept) mà client muốn tương tác, biểu diễn thông qua trạng thái (State) và các hành vi khả thi.

JJ Geewax (API Design Patterns - Ch.1): Định nghĩa API là "Giao diện hợp đồng chính xác giữa người cung cấp dịch vụ (Server/Provider) và người tiêu dùng (Consumer/Client)". Geewax phân biệt rõ giữa Interface (đặc tả) và Implementation (thực thi). Resource trong Geewax là "thực thể dữ liệu cơ sở có thể định danh duy nhất (identifiable discrete entity)", sở hữu vòng đời, thuộc tính và hỗ trợ các thao tác chuẩn hóa (CRUD).

2. Đối chiếu 5 đặc điểm trong Slide 05 với quan điểm Geewax & Higginbotham

Slide 05 đã nêu 5 đặc điểm cốt lõi của một API hiện đại: (1) Có hợp đồng rõ ràng (Contract-first), (2) Ẩn cài đặt (Abstraction), (3) Có thể tái sử dụng (Reusability), (4) Độc lập ngôn ngữ & nền tảng (Language-agnostic), (5) Có versioning & quan sát được (Versioned & Observable). Sau đây là 5 câu phân tích chuyên sâu đối chiếu với các bổ sung từ sách:

Đặc điểm Slide 05	Phân tích & Góc nhìn Tương đồng	Điểm bổ sung mới từ Higginbotham & Geewax
1. Hợp đồng rõ ràng (Contract-first)	Cả hai tác giả đều nhấn mạnh API phải có Spec (OpenAPI/Protobuf) trước khi code. Client không cần biết code backend mà chỉ cần đọc hợp đồng.	Bổ sung "Affordance & Predictability" (Higginbotham): Hợp đồng không chỉ là schema kiểu dữ liệu mà phải mang tính <i>dự đoán được (predictable)</i> và hướng nghiệp vụ (Job-to-be-done), chỉ dẫn rõ client có thể làm gì tiếp theo thay vì chỉ phơi bày DB schema.

2. Ẩn cài đặt (Abstraction)	Client không phụ thuộc vào việc Server đổi Postgres sang MongoDB hay Go sang Python.	Bổ sung "Loose Coupling & Domain Isolation" (Geewax): Geewax phân tích rằng abstraction tốt phải chống rò rỉ ranh giới nội bộ (Leak-free abstraction). Tuyệt đối không phơi bày các khóa ngoại (Foreign Keys) dạng DB nội bộ mà phải đóng gói thành các Resource References an toàn.
3. Có thể tái sử dụng (Reusability)	Một API phục vụ nhiều client khác nhau (Web, Mobile, 3rd-party) mà không sửa server code.	Bổ sung "Composability & API-as-a-Product" (Higginbotham): Không phải tái sử dụng thụ động mà API phải được thiết kế thành các khối lắp ghép (Composable building blocks), có tính nhất quán (Consistency) giữa các resource khác nhau trong cùng hệ sinh thái.
4. Độc lập ngôn ngữ & nền tảng (Language-agnostic)	Giao tiếp qua HTTP/JSON/Protobuf cho phép bất kỳ ngôn ngữ nào cũng tích hợp được.	Bổ sung "Ergonomics & Developer Experience (DX)" (Geewax): Độc lập ngôn ngữ là điều kiện cần; điều kiện đủ là sự <i>thân thiện lập trình (Ergonomics)</i> , bao gồm việc sinh SDK mượt mà, định dạng lỗi chuẩn hóa (RFC 7807/9457 Problem Details) để ngôn ngữ nào cũng parse lỗi đồng nhất.
5. Có versioning & quan sát được (Versioned & Observable)	API có định danh phiên bản (v1, v2) để nâng cấp không làm gãy client, kèm Logging, Tracing, Metrics.	Bổ sung "Backward Compatibility & Deprecation Policy" (Geewax & Higginbotham): Cả hai tác giả bổ sung nguyên tắc thiết kế <i>Tiến hóa không phá vỡ (Non-breaking evolution)</i> : Thêm trường mới được phép nhưng cấm đổi kiểu hoặc xóa trường cũ. Phải có chính sách thông báo khai tử (Deprecation lifecycle) rõ ràng trước khi bump major version.